

Memory Optimizations

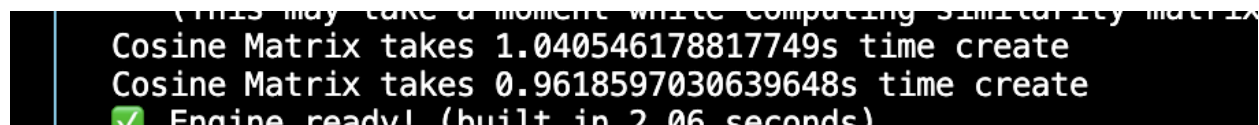
The amount of memory that I saved was 84.8%.

Before: 35.3198 MB

After: 5.368606MB

Performance Matrix Analysis

Most of the time, it took around 1 second to create the cosine similarity matrix



```
(THIS may take a moment while computing similarity matrix)
Cosine Matrix takes 1.040546178817749s time create
Cosine Matrix takes 0.9618597030639648s time create
✔ Engine ready! (built in 2.06 seconds)
```

For it to run that fast considering we are using a dataset of size (16180, 9) is very good runtime. The cosine similarity takes the features matrix, which takes the shape (number of books, number of genre features + 2). This means that the function could take more time if the number of books and number of genre features we are looking at increases over time. That is why doing memory optimization of the data types is so important to reduce runtime.

Algorithm Comparison:

Content - Based:

- Strengths: Using the content of the books allows for more accurate books to be selected for recommendations.
- Weaknesses: This approach doesn't take into account popularity, so the books selected may not be good and thus the user might be more discouraged from reading the book. The recommended books might be too similar to the original, which could lead to a lack of variety being shown towards the user.

Popular - Based:

- Strengths: Recommends very popular genre books to individuals, potentially making them more interested in reading
- Weaknesses: Very simple approach with no specifics used. Most range in scores. We want the range to be high and precise, with low range.

Hybrid - Based:

- Strengths: Utilizes both popularity and content to recommend books that are both popular and that have similar content. This allows the user to potentially read books that are high quality, but also have a lot of variety.
- Weaknesses: Lesser match scores (recommendations score). This could be good however because it could lead the user to have more variety in reading rather than reading the

same types of books. This could be bad however if the user wants exactly the same type of books.

Design Decisions

How I calculated popularity score:

```
def _calculate_popularity(self) -> pd.Series:
    """
    Calculate a popularity score for each book.

    TODO: Design a popularity metric
    Consider both the number of ratings and the average rating

    Week 10 concept: Use vectorized operations for efficiency!

    Returns:
    | Series of popularity scores
    """

    # Normalize num_ratings to 0-1 scale first
    max_ratings = self.data['num_ratings'].max()
    normalized_ratings = self.data['num_ratings'] / max_ratings

    # Weight both factors
    popularity = normalized_ratings * (self.data['average_rating'] / 5.0)

    return popularity
```

I found the max rating, then normalized it and weighted it by multiplying with the (average rating / 2).

How I weighted Features in Content-Based filtering:

```
def _prepare_features(self):
    """
    Prepare feature vectors for similarity computation.

    TODO: Create feature vectors from book metadata
    - One-hot encode genres (use MultiLabelBinarizer)
    - Consider author, page count, ratings

    Week 10 concept: Vectorization for efficiency
    """

    # TODO: Create genre feature matrix
    # Hint: Use MultiLabelBinarizer to convert list of genres to binary matrix
    mlb = MultiLabelBinarizer()
    genre_features = mlb.fit_transform(self.books_df['genres'])

    # TODO: Add normalized numerical features
    # Normalize page count and ratings to 0-1 scale
    normalized_pages = (
        | self.books_df['num_pages'] / self.books_df['num_pages'].max()
        )
    normalized_rating = self.books_df['average_rating'] / 5.0

    # Combine features
    # Shape: (n_books, n_genre_features + 2)
    self.feature_matrix = np.hstack([
        genre_features,
        normalized_pages.values.reshape(-1, 1),
        normalized_rating.values.reshape(-1, 1)
    ])
```

I one hot encoded the genre features, then normalized the number of pages and ratings value. I use that feature matrix to create a cosine similarity matrix to get recommendations.

How I combined Strategies in Hybrid-based Recommendations:

```

# TODO: Get content-based recommendations
content_recs = self.content_recommender.recommend(book_id, n * 3)

# TODO: Create a score dictionary for efficient lookup
content_scores = {book_id: score for book_id, _, score in content_recs}

# TODO: Normalize popularity scores for the candidate books
candidate_ids = [book_id for book_id, _, _ in content_recs]
candidate_books = self.books_df[self.books_df['book_id'].isin(candidate_ids)]

max_pop = candidate_books['popularity_score'].max()

# TODO: Combine scores
combined_scores = []
for _, row in candidate_books.iterrows():
    book_id = row['book_id']
    content_score = content_scores.get(book_id, 0)
    popularity_score = row['popularity_score'] / max_pop if max_pop > 0 else 0

    # Weighted combination
    combined = (self.content_weight * content_score +
                self.popularity_weight * popularity_score)

    combined_scores.append((
        book_id,
        row['book_title'],
        combined
    ))

# TODO: Sort by combined score and return top N
combined_scores.sort(key=lambda x: x[2], reverse=True)
return combined_scores[:n]

```

I created a combine score that would both combine the content score and popularity score through this equation: $(\text{content_weight} * \text{content_score}) + (\text{popularity_weight} * \text{popularity_score})$. This way, the recommendation scores are now taking into account books that are similar, but have a good popularity score. We want to recommend books to the user that are similar, but also popular.

Recommendations Quality

I compared the recommendations systems with for finding books similar to the Hunger Games

Here were the results:

Comparing recommendation strategies for: 'The Hunger Games'		
Strategy: CONTENT		
1. Crossed	Score: 1.000	
2. Extras	Score: 1.000	
3. The Bar Code Tattoo	Score: 1.000	
4. Matched	Score: 1.000	
5. Reached	Score: 1.000	
Time: 0.0038s		
Strategy: POPULARITY		
1. Harry Potter and the Sorcerer's Stone	Score: 0.894	
2. Harry Potter and the Sorcerer's Stone	Score: 0.894	
3. The Hunger Games	Score: 0.748	
4. The Hunger Games	Score: 0.748	
5. To Kill a Mockingbird	Score: 0.519	
Time: 0.0032s		
Strategy: HYBRID		
1. Divergent	Score: 0.999	
2. Matched	Score: 0.751	
3. Uglies	Score: 0.746	
4. Delirium	Score: 0.733	
5. Pretties	Score: 0.718	
Time: 0.0040s		

From the results, we see books like Divergent, Harry Potter, To Kill a Mockingbird, and The Bar Code Tattoo show up. These books are all very similar to The Hunger Games so the

recommendations look to be of good quality. What is interesting is that content-based had the highest match scores. That may be good in terms of recommendations, but bad for the user itself: less diversity of the type of books within the same genre or themes could become boring to read.